

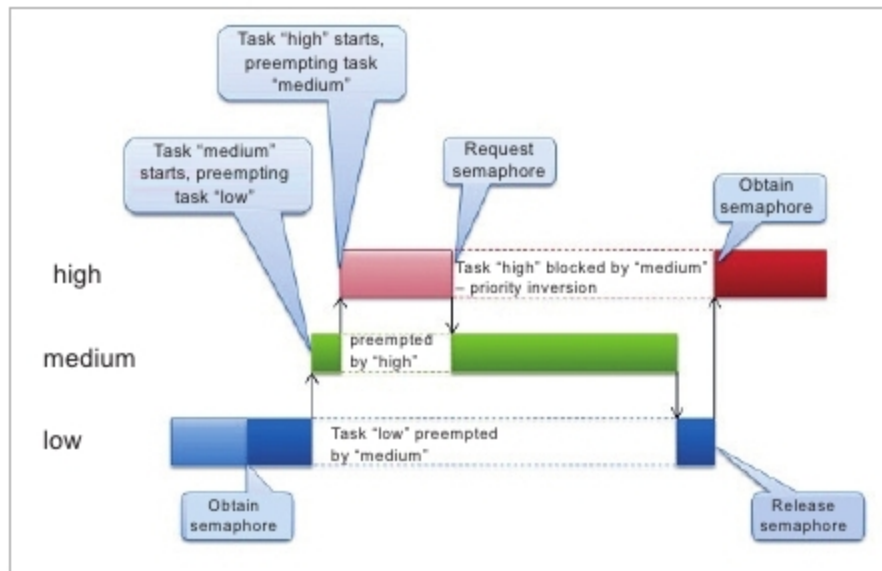


Figure 1: Priority inversion

common.h includes all the header files and functions that are common to *PriInvLow.c*, *PriInvMed.c* and *PriInvHigh.c*.

The *gettime* function returns the current time. It is used in the function *doSomething*, which provides a delay of exactly 'n' seconds. This is considered as a delay for a process to complete its critical section operation. So, in order to understand when a process is going to start and end the critical section, we keep track of the start time and the end time for this function. Here, the *sched_getparam* returns the scheduling parameters for the calling process. Using this we can access the priority of the current running process.

PriInvLow.c*, *PriInvHigh.c* and *PriInvMed.c

Lines 7 to 15 in programs *PriInvLow.c*, *PriInvHigh.c* and *PriInvMed.c* are used to set parameters for the respective processes and, hence, kept common in all three programs with just a change in priority value. Function *sched_setaffinity* makes the process run on a dedicated CPU only, which is mentioned through the CPU affinity mask. In this case, we set this value to 1 for all three processes to make sure that all of them share a single processor. Line No.13 in the priority is assigned to that process. In the case of *PriInvLow.c*, the priority is 20; for *PriInvHigh.c*, the priority is 38; and for *PriInvMed.c*, it is 28.

In the case of *PriInvLow.c* and *PriInvHigh.c*, a key is again generated with the same parameters as that of *sem.c* in order to get the lock of the created semaphore.

The parameters of *struct sembuf* are as shown below:

```
struct sembuf {
    ushort sem num; /* semaphore index in array */
    short sem op; /* semaphore operation */
    short sem flg; /* operation flags */
};
```

Here, *sem num* indicates the number of the semaphore in the set on which the operation has to be performed. *sem op* can be positive, negative or zero. If *sem op* is negative, the value will be subtracted from the semaphore and the resource is locked. Other calling processes for those resources will go

to sleep. If *sem op* is positive, then the value will be added to the semaphore and the resource is unlocked. If *sem op* is zero, the calling process will sleep until the value is zero.

The *sem flg* indicates the operation flag. It takes the value *IPC NOWAIT* and *SEM UNDO*. If the semaphore is locked, *sem flg* is *IPC NOWAIT*. If the process wants to acquire a semaphore, instead of actually doing so, it will immediately return -1 and *errno == EAGAIN*. If *sem flg* is *SEM UNDO*, then the semaphore is undone on exiting the process.

In *PriInvLow.c* and *PriInvHigh.c*, the values are set for the *sembuf* buffer. *semget* will return the ID related to the key. This ID and *sembuf* will be used by the *semop* function to lock and unlock the semaphore. This locking and unlocking operation is performed in *PriInvLow.c* and *PriInvHigh.c* before and after calling the *doSomething* function, respectively.

However, in *PriInvMed.c*, no such locking and unlocking is performed either before or after calling the *doSomething* function.

Different CPU affinities for processes

The sets of codes used for these sections are exactly the same as that of the code mentioned for the 'Priority inversion with a semaphore' section, except for a different CPU affinity for each process.

In all three programs – *High.c*, *Low.c* and *Medium.c* – Line No. 9 represents the affinity set for that process. The affinity for the program *High.c* is set to 2, *Low.c* is set to 0 and *Medium.c* is set to 1.

Priority inheritance with PI-futex

Futex is short for 'fast user space Mutex'. It is used for efficient user space locking. The futex mechanism is a fast, lightweight kernel-assisted locking primitive for user space applications. Futex is an unsigned 32-bit integer on all platforms. It provides for very fast uncontended lock acquisition and release, as the futex state is stored in a user space variable. Atomic operations are used in order to change the state of the futex in the uncontended case without the overhead of a syscall. In contended cases, the kernel is invoked to put tasks to sleep and to wake them up.

```
int futex(int uaddr, int op, int val, const struct timespec
timeout, int uaddr2, int val3)
```

The 'uaddr' represents the address of futex, and which action is to be performed depends on the 'op' argument. Other parameters can be used depending on the required operation. There is no *glibc* wrapper for this system call; hence the *syscall* function is used.

The two most commonly used operations are *FUTEX WAIT* and *FUTEX WAKE*.

FUTEX WAIT: The kernel checks the value at *uaddr* with *val*; then if both are the same, it blocks the calling thread/process. The last two parameters are ignored for this operation